WORKATO GUIDE

Revenue Orchestration (Approval-Gated) — Workato Implementation Guide

Workflow ID: 32-revenue-orchestration • Backstory public rollout asset

This file is a plain-English implementation guide. It is not an importable JSON artifact.

Workato moves reusable automation between workspaces as recipe packages, and package import uses a `.zip` file in Recipe Lifecycle Management. Imported connections arrive as placeholders and must be authenticated in the destination workspace. Build this workflow in Workato as recipes, recipe functions, connectors, properties, and tables, then export/import the package when you need to promote it between environments.

Workflow Summary

- Workflow ID: `32-revenue-orchestration`
- Category: pipeline-forecasting
- Source trigger in this catalog: Webhook — external signal with Slack approval gate |
- Recommended Workato trigger surface: API Platform endpoint or native app event trigger
- Delivery targets: Slack channel or direct message

What To Create In Workato

- Backstory custom connector built with the Connector SDK
- Slack connector or Workbot for Slack for channel and DM delivery
- Lookup tables, data tables, or project properties for routing and canonical maps

Build Order

1. Create or choose the target project and folder where this workflow will live.
2. Build or install the Backstory custom connector in Connector SDK. If you have an OpenAPI 3.0 definition, Workato supports using that as a starting point in the Connector SDK.
3. Create the Recipe Function `normalize_run_context` so reusable logic is not trapped inside one large recipe.

4. Create the Recipe Function `load_source_records` so reusable logic is not trapped inside one large recipe.
5. Create the Recipe Function `assemble_business_context` so reusable logic is not trapped inside one large recipe.
6. Create the Recipe Function `render_delivery_payload` so reusable logic is not trapped inside one large recipe.
7. Create the Recipe Function `record_run_summary` so reusable logic is not trapped inside one large recipe.
8. Build the primary recipe `32_revenue_orchestration_primary_recipe` with the trigger surface API Platform endpoint or native app event trigger.
9. Store routing defaults, destination IDs, and mode switches in connections, environment properties, project properties, lookup tables, or data tables.
10. Test the recipe and each function with representative payloads, then export the project as a Workato package `.zip` if you need to move it to QA or production.

Recipe Functions To Create

- `normalize_run_context` : Normalize the Revenue Orchestration (Approval-Gated) trigger into the shared run_context contract. Inputs: trigger_payload, workflow defaults. Outputs: run_context. Native features: Recipe function by Workato, Formula mode, Variables.
- `load_source_records` : Load and normalize the primary Revenue Orchestration (Approval-Gated) business inputs into source_record objects. Inputs: run_context. Outputs: source_record[]. Native features: Recipe function by Workato, Backstory custom connector action, Lookup tables or data tables.
- `assemble_business_context` : Join the intermediate context needed for Revenue Orchestration (Approval-Gated), especially Load Account Context, Wait For Decision. Inputs: source_record[], run_context. Outputs: source_record[], enrichment_context candidates. Native features: Lists, Object actions, Data tables.
- `render_delivery_payload` : Render normalized delivery_payload objects for Revenue Orchestration (Approval-Gated). Inputs: structured_ai_json, source_record, run_context. Outputs: delivery_payload. Native features: Recipe function by Workato, Object construction, Formula mode.
- `record_run_summary` : Capture deterministic summary, dedupe, and retry state for Revenue Orchestration (Approval-Gated). Inputs: run_context, delivery results. Outputs: execution summary. Native features: Variables, Data tables.

Primary Recipe Outline

1. Call normalize_run_context
2. Use Backstory custom connector action for Load Account Context
3. Use an LLM connector or AI step to return strict JSON for Draft Proposed Action
4. Use Slack connector for Request Approval
5. Use Backstory custom connector action for Wait For Decision

6. Use Slack connector for Notify Outcome
7. Record deterministic run summary and retry state

Contracts To Preserve

- `run_context`: workflow-level execution context such as trigger, mode, lookback, delivery mode, and dry-run state.
- `source_record`: normalized business record from the source adapter or source-system action layer.
- `enrichment_context`: MCP or native app enrichment results used only during synthesis.
- `delivery_payload`: deterministic delivery fields for the final native connector or app action.

Structured AI Requirements

- `draft_proposed_action`: return JSON only with keys `title`, `summary`, `confidence`, `recommended_actions`, `source_refs`. Intent: Uses the model to generate a structured CRM update plus owner-facing follow-up proposal.

Validation Checklist

- No repeated Universal HTTP or custom-action sprawl for Slack, calendar, CRM, or email delivery when a native connector exists.
- All secrets live in connections, environment properties, or project properties rather than formula literals.
- LLM or agent output is validated as structured JSON before any delivery or persistence step runs.
- Recipe Functions, lookup tables, and data tables own reuse, routing, and dedupe instead of large inline scripts.
- If this build is moved between workspaces, export and import it as a Workato package `.zip`, then reconnect placeholder connections in the target workspace.

Official References

- Recipe lifecycle management
- Importing packages
- Recipe function by Workato
- Using the Workato Connector SDK
- Slack connector
- Google Calendar connector